HUST

ĐẠI HỌC BÁCH KHOA HÀ NỘI

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.

ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

FUNDAMENTALS OF OPTIMIZATION

Integer Linear Programming

ONE LOVE. ONE FUTURE.

CONTENT

- **Introduction to integer programming**
  - **Relaxation and bound**
  - Branch and bound
  - Cutting plane
  - Integer rounding
  - Gomory cut
  - Schema of branch-and-cut algorithm
  - OR-Tools
- Some techniques to linearize a constraint
- Exercises
  - Balanced class teacher assignment
  - Blood type correction problem
  - TSP
  - Facility location
  - MultiCast Routing Problem
  - Capacitated Vehicle Routing Problem

RELAXATION AND BOUND

- Given an Integer Program (IP)
- Find decreasing sequence of upper bounds

$$\overline{z_1} > \overline{z_1} > \ldots > \overline{z_s} \geq z$$

- Find increasing sequence of lower bounds

$$\underline{z_1} < \underline{z_1} < \ldots < \underline{z_t} \leq z$$

- Algorithm stop when $\overline{z_s} - \underline{z_t} \leq \varepsilon$

RELAXATION AND BOUND

- Primal bounds
  - Every feasible solution $x^* \in X$ provides a lower bound of the maximization problem: $\underline{z} = cx^* \leq z$
  - Example: in TSP, every closed tour is a upper bound of the objective function (as TSP is a minimization problem)

RELAXATION AND BOUND

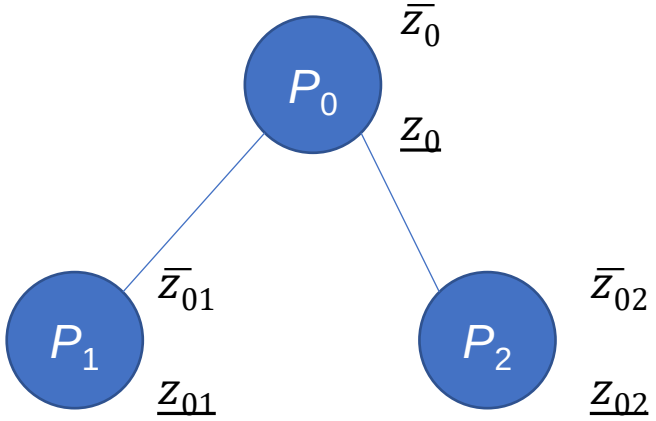
- Dual bounds
  - Finding upper bounds for a maximization problem (or lower bounds for a minimization problem) gives dual bounds of the objective
- **Definition** A problem (RP) $z^R = \max\{f(x): x \in T \subseteq R^n\}$ is a relaxation of (IP) $z = \max\{c^T x: x \in X \subseteq Z^n\}$ if:
  - $X \subseteq T$
  - $f(x) \geq c^T x, \forall\, x \in X$

RELAXATION AND BOUND

- Linear Relaxation
  - $Z^{LP} = \max\{c^T x: x \in P\}$ with $P = \{x \in R^n: Ax \leq b\}$ is a linear relaxation program of the (IP) $\max\{c^T x: x \in P \cap Z^n\}$

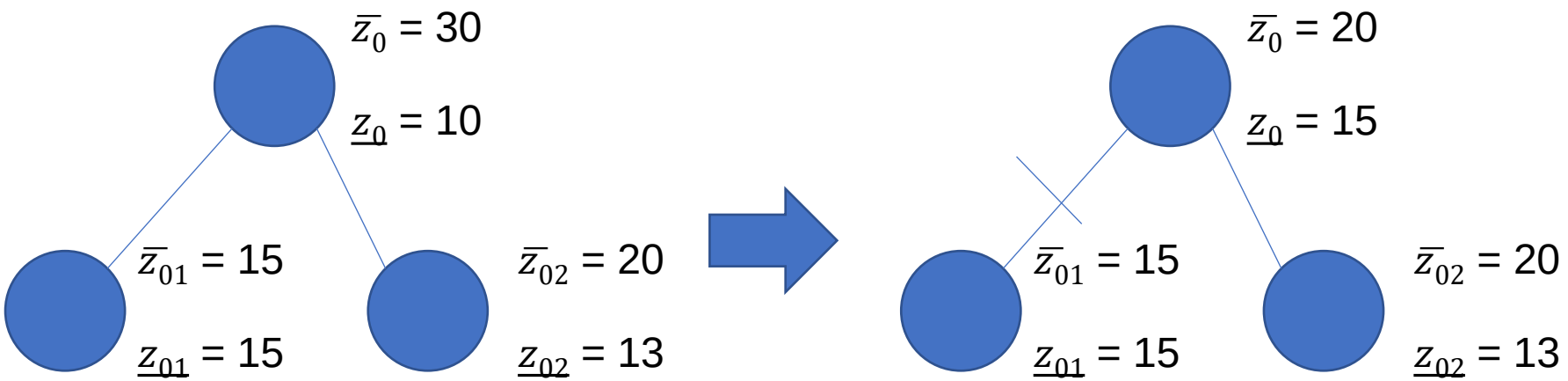
BRANCH AND BOUND

- Feasible region of P_0 is divided into feasible regions of P_1 and P_2: $X(P_0) = X(P_1) \cup X(P_2)$



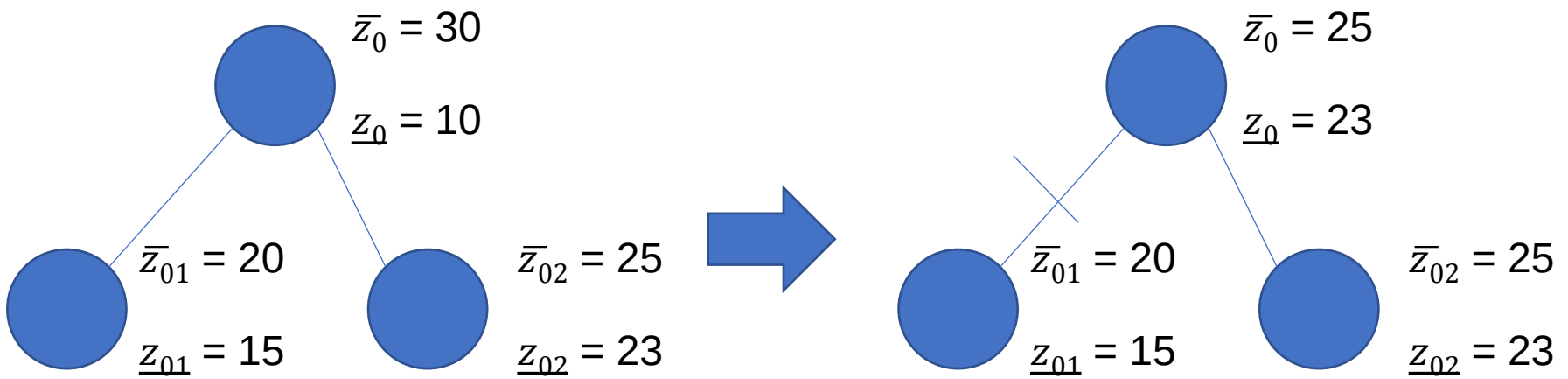
BRANCH AND BOUND

- Feasible region of P_0 is divided into feasible regions of P_1 and P_2: $X(P_0) = X(P_1) \cup X(P_2)$



BRANCH AND BOUND

- Feasible region of P_0 is divided into feasible regions of P_1 and P_2: $X(P_0) = X(P_1) \cup X(P_2)$



BRANCH AND BOUND

Initial problem S with formulation P on a list L;

Incumbent x^* is initialized with primal bound $\underline{z} = -INF$;

while L not empty do {

  Select a problem S^i with formulation P^i from L;

  Solve LP relaxation over P^i got dual bound $\overline{z}^i$ and solution $x^i(LP)$;

  if $\overline{z}^i \leq \underline{z}$ then continue; // prune by dual bound;

  if $x^i(LP)$ integer then{

    $\underline{z} = \overline{z}^i$;

    $x^* = x^i(LP)$;

  }else{

    select a component x_i of $x^i(LP)$ whose value λ_i is fractional;

    $P_1^i = P^i \cup (x_i \leq \lfloor \lambda_i \rfloor)$, $P_2^i = P^i \cup (x_i \geq \lceil \lambda_i \rceil)$;

    add P_1^i and P_2^i to L;

  }

}

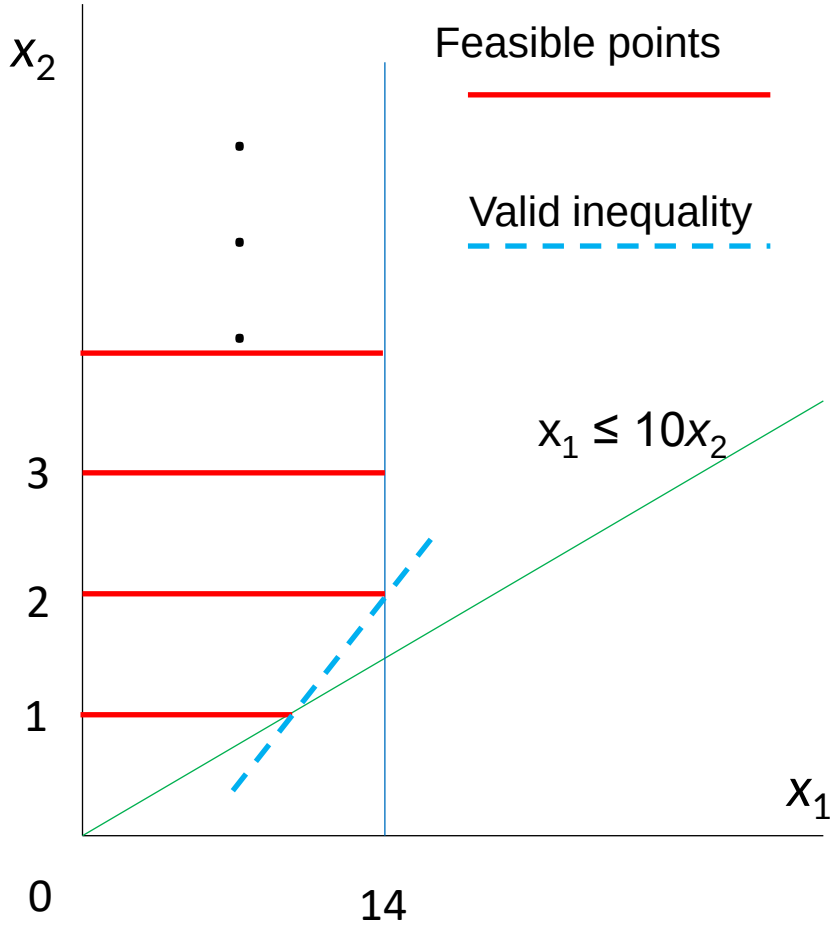
return x^*;

CUTTING PLANE

- Given a (MIP) $\max\{c^T z: z \in X\}$
- Inequality $\pi z \leq \pi_0$ is called a valid inequality if $\pi z \leq \pi_0$ is true for all $z \in X$
- Finding valid inequalities allows us to narrow the search space, transform the (MIP) to a corresponding (LP) in which an optimal solution to (LP) is an optimal solution to the original (MIP)

CUTTING PLANE

- Example, consider a MIP with $X = \{(x_1,x_2): x_1 \le 10x_2, 0 \le x_1 \le 14, x_2 \in Z_+^1\}$
- Red lines represent X
- $x_1 \le 6 + 4x_2$ is a valid inequality (dashed line)



CUTTING PLANE

- Example: **Integer Rounding**
  - Consider feasible region $X = P \cap Z^3$ where $P = \{x \in R_+^3 : 5x_1 + 9x_2 + 13x_3 \geq 19\}$
  - From $5x_1 + 9x_2 + 13x_3 \geq 19$ we have $x_1 + \frac{9}{5}x_2 + \frac{13}{5}x_3 \geq \frac{19}{5}$

  → $x_1 + 2x_2 + 3x_3 \geq \frac{19}{5}$
  - As x_1, x_2, x_3 are integers, so we have

$$x_1 + 2x_2 + 3x_3 \geq \lceil \frac{19}{5} \rceil = 4 \text{ (this is a valid inequality for } X)$$

GOMORY CUT

- (IP) max $\{cx: Ax = b, x \geq 0 \text{ and integer}\}$
- Solve corresponding linear programming relaxation (*LP*) max $\{cx: Ax = b, x \geq 0\}$
- Suppose with an optimal basis, the (LP) is rewritten in the form

$$\overline{a_{00}} + \sum_{j \in JN} \overline{a_{0j}} x_j \rightarrow \max$$

$$x_{B_u} + \sum_{j \in JN} \overline{a_{uj}} x_j = \overline{a_{u0}},\ u = 1, 2, \ldots, m$$

$$x \geq 0 \text{ and integer}$$

with $\overline{a_{0j}} \leq 0$ (as these coefficients corresponds to a maximizer), and $\overline{a_{u0}} \geq 0$

GOMORY CUT

- (IP) max {cx: $Ax = b$, $x \geq 0$ and integer}
- Solve corresponding linear programming relaxation (*LP*) max {cx: $Ax = b$, $x \geq 0$}
- Suppose with an optimal basis, the (LP) is rewritten in the form

$$\overline{a_{00}} + \sum_{j \in JN} \overline{a_{0j}} x_j \rightarrow \max$$

$$x_{B_u} + \sum_{j \in JN} \overline{a_{uj}} x_j = \overline{a_{u0}},\ u = 1, 2, \ldots, m$$

$$x \geq 0 \text{ and integer}$$

with $\overline{a_{0j}} \leq 0$ (as these coefficients corresponds to a maximizer), and $\overline{a_{u0}} \geq 0$

- If the basic optimal solution x^* is not integer, then there exists some row u with $\overline{a_{u0}}$ is not integer

→ Create a Gomory cut $x_{B_u} + \sum_{j \in JN} \lfloor \overline{a_{uj}} \rfloor x_j \leq \lfloor \overline{a_{u0}} \rfloor$ (1)

GOMORY CUT

- **Example**. Consider the Integer Linear Program (ILP)

$$f(x_1, x_2, x_3, x_4, x_5) = x_1 + x_2 \rightarrow \max$$

$$2x_1 + x_2 + x_3 = 8$$

$$3x_1 + 4x_2 + x_4 = 24$$

$$x_1 - x_2 + x_5 = 2$$

$$x_1, x_2, x_3, x_4, x_5 \geq 0 \text{ and integer}$$

GOMORY CUT

- **Solve the corresponding** (LP)

$$f(x_1, x_2, x_3, x_4, x_5) = x_1 + x_2 \rightarrow \max$$

$$2x_1 + x_2 + x_3 = 8$$

$$3x_1 + 4x_2 + x_4 = 24$$

$$x_1 - x_2 + x_5 = 2$$

$$x_1, x_2, x_3, x_4, x_5 \geq 0$$

→ Optimal solution (1.6, 4.8, 0, 0, 5.2) with J_B = (1,2,5) and J_N = (3, 4)

Rewrite the original (ILP)

$$f(x_1, x_2, x_3, x_4, x_5) = 6.4 - 0.2x_3 - 0.2x_4 \rightarrow \max$$

$$x_1 + 0.8x_3 - 0.2x_4 = 1.6$$

$$x_2 - 0.6x_3 + 0.4x_4 = 4.8$$

$$x_5 - 1.4x_3 + 0.6x_4 = 5.2$$

$$x_1, x_2, x_3, x_4, x_5 \geq 0 \text{ and integer}$$

GOMORY CUT

- Consider $x_1 + 0.8x_3 - 0.2x_4 = 1.6$

→$x_1 + 0x_3 - x_4 \leq \lfloor 1.6 \rfloor = 1$

→Add slack variable x_6 and constraint (Gomory cut): $0.8x_3 + 0.8x_4 - x_6 = 0.6$

- Consider $x_2 - 0.6x_3 + 0.4\,x_4 = 4.8$

→$x_2 - x_3 + 0.x_4 \leq \lfloor 4.8 \rfloor = 4$

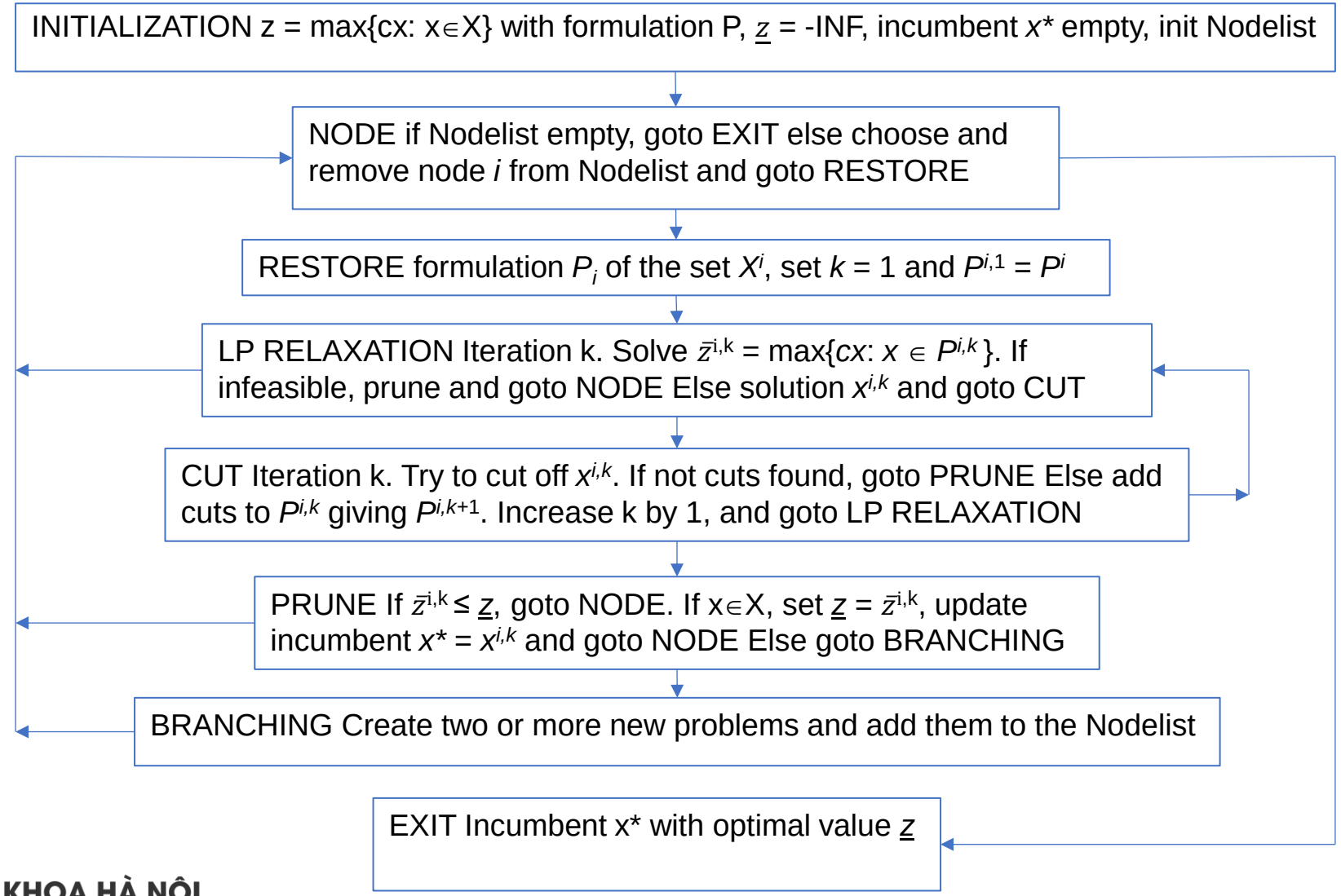
→Add slack variable x7 and constraint (Gomory cut): $0.4x_3 + 0.4x_4 - x_7 = 0.8$

- Consider $x_5 - 1.4x_3 + 0.6x_4 = 5.2$

→$x_5 - 2x_3 + 0x_4 \leq \lfloor 5.2 \rfloor = 5$

→Add slack variable x8 and constraint (Gomory cut) $0.6x_3 + 0.6x_4 - x_8 = 0.2$

BRANCH AND CUT [Wolsey, 98]



Example of Branch-and-cut algorithm

- **Example**: Maximize $x_1 + x_2$ subject to the following constraints
  1. $x_1 \leq 10x_2 + 7$
  2. $2x_1 + 3x_2 \leq 20$
  3. $0 \leq x_1 \leq 14$,
  4. $0 \leq x_2 \leq 20$,
  5. $x_1 \in R$,
  6. $x_2 \in Z$

- **Solving**: The linear relaxation of the MIP as follows: Maximize $x_1 + x_2$ subject to the following constraints
  1. $x_1 \leq 10x_2 + 7$
  2. $2x_1 + 3x_2 \leq 20$
  3. $0 \leq x_1 \leq 14$,
  4. $0 \leq x_2 \leq 20$,
  5. $x_1 \in R$,
  6. $x_2 \in R$

Example of Branch-and-cut algorithm

- **Solving**: The linear relaxation of the MIP as follows: Maximize $x_1 + x_2$ subject to the following constraints
  1. $x_1 \leq 10x_2 + 7$
  2. $2x_1 + 3x_2 \leq 20$
  3. $0 \leq x_1 \leq 14$,
  4. $0 \leq x_2 \leq 20$,
  5. $x_1 \in R$,
  6. $x_2 \in R$

➔Solution obtained:

$x_1 = 9.608695652173914$,

$x_2 = 0.2608695652173912 4$

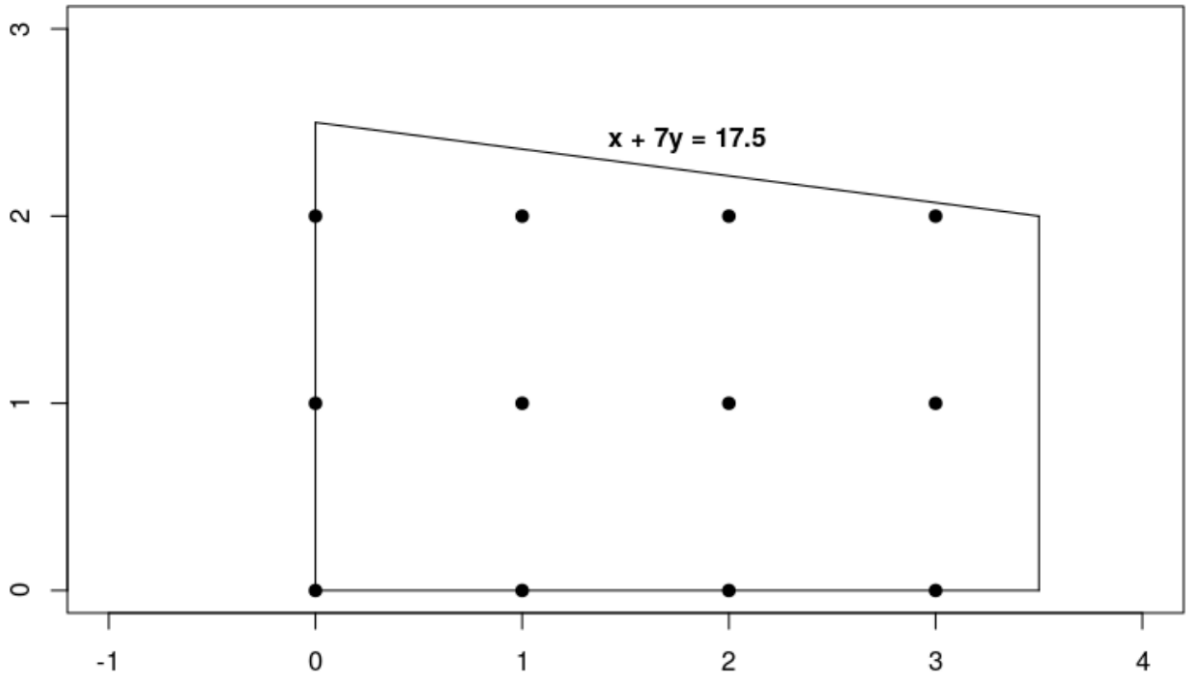
Example of Branch-and-cut algorithm

- Branching on x_2 results to two following Linear Programs
- ***LP1***: Maximize $x_1 + x_2$ subject to the following constraints
  1. $x_1 \leq 10x_2 + 7$
  2. $2x_1 + 3x_2 \leq 20$
  3. $0 \leq x_1 \leq 14$,
  4. $0 \leq x_2 \leq 20$,
  5. $x_1 \in R$,
  6. $x_2 \in R$
  7. $x_2 \leq 0$
- ***LP2***: Maximize $x_1 + x_2$ subject to the following constraints
  1. $x_1 \leq 10x_2 + 7$
  2. $2x_1 + 3x_2 \leq 20$
  3. $0 \leq x_1 \leq 14$,
  4. $0 \leq x_2 \leq 20$,
  5. $x_1 \in R$,
  6. $x_2 \in R$
  7. $x_2 \geq 1$

Example of Branch-and-cut algorithm

- Solving LP1 to obtain the optimal solution $x_1 = 7, x_2 = 0$. This is a feasible solution to the original problem since x_2 is assigned to an integer. The value of objective function is 7. The algorithm sets this solution as incumbent and the best solution value $f^* = 7$
- Solving LP2 to obtain the optimal solution $x_1 = 8.5, x_2 = 1$. This is a feasible solution to the original problem since x_2 is assigned to an integer. The value of objective function is 9.5. This solution is better than the current incumbent, so the incumbent is updated to this solution, and the best solution value f^* is updated to 9.5.
- In conclusion, the original problem has the optimal solution of $x_1 = 8.5, x_2 = 1$, and the value of the objective function is 9.5.

- **Example 1**: Maximize $x + 10y$ subject to the following constraints:
  1. $x + 7y \leq 17.5$
  2. $0 \leq x \leq 3.5$
  3. $0 \leq y$
  4. x, y are integers
- Geographic approach



OR-Tools

To solve a MIP problem, your program should include the following steps:

1. Import the linear solver wrapper,
2. declare the MIP solver,
3. define the variables,
4. define the constraints,
5. define the objective,
6. call the MIP solver and
7. display the solution

OR-Tools

```python
from ortools.linear_solver import pywraplp


def main():
    # Create the mip solver with the CP-SAT backend.
    solver = pywraplp.Solver.CreateSolver("SAT")
    if not solver:
        return

    infinity = solver.infinity()
    # x and y are integer non-negative variables.
    x = solver.IntVar(0.0, infinity, "x")
    y = solver.IntVar(0.0, infinity, "y")

    print("Number of variables =", solver.NumVariables())

    # x + 7 * y <= 17.5.
    solver.Add(x + 7 * y <= 17.5)

    # x <= 3.5.
    solver.Add(x <= 3.5)

    print("Number of constraints =", solver.NumConstraints())
    # Maximize x + 10 * y.
    solver.Maximize(x + 10 * y)

    print(f"Solving with {solver.SolverVersion()}")
    status = solver.Solve()

    if status == pywraplp.Solver.OPTIMAL:
        print("Solution:")
        print("Objective value =", solver.Objective().Value())
        print("x =", x.solution_value())
        print("y =", y.solution_value())
    else:
        print("The problem does not have an optimal solution.")

    print("\nAdvanced usage:")
    print(f"Problem solved in {solver.wall_time():d} milliseconds")
    print(f"Problem solved in {solver.iterations():d} iterations")
    print(f"Problem solved in {solver.nodes():d} branch-and-bound nodes")


if __name__ == "__main__":
    main()
```

Example 2: Maximize $7x_1 + 8x_2 + 2x_3 + 9x_4 + 6x_5$ subject to the following constraints:

1. $5\,x_1 + 7\,x_2 + 9\,x_3 + 2\,x_4 + 1\,x_5 \leq 250$
2. $18\,x_1 + 4\,x_2 - 9\,x_3 + 10\,x_4 + 12\,x_5 \leq 285$
3. $4\,x_1 + 7\,x_2 + 3\,x_3 + 8\,x_4 + 5\,x_5 \leq 211$
4. $5\,x_1 + 13\,x_2 + 16\,x_3 + 3\,x_4 - 7\,x_5 \leq 315$

OR-Tools

```
from ortools.linear_solver import pywraplp


def create_data_model():
    """Stores the data for the problem."""
    data = {}
    data["constraint_coeffs"] = [
        [5, 7, 9, 2, 1],
        [18, 4, -9, 10, 12],
        [4, 7, 3, 8, 5],
        [5, 13, 16, 3, -7],
    ]
    data["bounds"] = [250, 285, 211, 315]
    data["obj_coeffs"] = [7, 8, 2, 9, 6]
    data["num_vars"] = 5
    data["num_constraints"] = 4
    return data
```

```
def main():
    data = create_data_model()
    # Create the mip solver with the SCIP backend.
    solver = pywraplp.Solver.CreateSolver("SCIP")
    if not solver:
        return

    infinity = solver.infinity()
    x = {}
    for j in range(data["num_vars"]):
        x[j] = solver.IntVar(0, infinity, "x[%i]" % j)
    print("Number of variables =", solver.NumVariables())

    for i in range(data["num_constraints"]):
        constraint = solver.RowConstraint(0, data["bounds"][i], "")
        for j in range(data["num_vars"]):
            constraint.SetCoefficient(x[j], data["constraint_coeffs"][i][j])
    print("Number of constraints =", solver.NumConstraints())
    # In Python, you can also set the constraints as follows.
    # for i in range(data['num_constraints']):
    #  constraint_expr = \
    # [data['constraint_coeffs'][i][j] * x[j] for j in range(data['num_vars'])]
    #  solver.Add(sum(constraint_expr) <= data['bounds'][i])
```

OR-Tools

```
    objective = solver.Objective()
    for j in range(data["num_vars"]):
        objective.SetCoefficient(x[j], data["obj_coeffs"][j])
    objective.SetMaximization()
    # In Python, you can also set the objective as follows.
    # obj_expr = [data['obj_coeffs'][j] * x[j] for j in range(data['num_vars'])]
    # solver.Maximize(solver.Sum(obj_expr))

    print(f"Solving with {solver.SolverVersion()}")
    status = solver.Solve()

    if status == pywraplp.Solver.OPTIMAL:
        print("Objective value =", solver.Objective().Value())
        for j in range(data["num_vars"]):
            print(x[j].name(), " = ", x[j].solution_value())
        print()
        print(f"Problem solved in {solver.wall_time():d} milliseconds")
        print(f"Problem solved in {solver.iterations():d} iterations")
        print(f"Problem solved in {solver.nodes():d} branch-and-bound nodes")
    else:
        print("The problem does not have an optimal solution.")


if __name__ == "__main__":
    main()
```

CONTENT

- Introduction to integer programming
  - Relaxation and bound
  - Branch and bound
  - Cutting plane
  - Integer rounding
  - Gomory cut
  - Schema of branch-and-cut algorithm
  - OR-Tools
- **Some techniques to linearize a constraint**
- Exercises
  - Balanced class teacher assignment
  - Blood type correction problem
  - TSP
  - Facility location
  - MultiCast Routing Problem
  - Capacitated Vehicle Routing Problem

SOME TECHNIQUES TO LINEARIZE A CONSTRAINT

- If then constraint
- Multiplication of Binary Variables
- Multiplication of Binary and Continuous Variables
- Maximum/Minimum Operators
- Absolute Value Function

If Then

- $x, y \in \{0, 1\}$ If $x = 1$ then $y = 1$ otherwise nothing
- Scenarios:
  - $x = 1, y = 1$
  - $x = 0, y = 0$
  - $x = 0, y = 1$
- Transformation:
  - $(x - 1) \times M + 1 \leq y$, in which $M \geq 2$
- Applications in TSP: x_{ij} is a binary variable to indicated that the salesman goes directly from the city i to the city j or not; and y_i is the city that the salesman visits at i^{th} time. The subtour : $if\ x_{ij} = 1\ then\ y_j = y_i + 1$
  - $(1 - x_{ij}) \times M + y_j \geq y_i + 1$
  - $(x_{ij} - 1) \times M + y_j \leq y_i + 1$

  in which M is a very large number

Multiplication of Binary Variables

- Non-linear constraint: $z = x \times y$ with $x, y \in \{0, 1\}$
- Transformation:
  - $z \leq x$
  - $z \leq y$
  - $x + y \leq z + 1$

x	y	$x \cdot y$	Constraints	Imply
0	0	0	$z \leq 0$ $z \leq 0$ $z \geq -1$ $z \in \{0,1\}$	$z = 0$
0	1	0	$z \leq 0$ $z \leq 1$ $z \geq 0$ $z \in \{0,1\}$	$z = 0$
1	0	0	$z \leq 1$ $z \leq 0$ $z \geq 0$ $z \in \{0,1\}$	$z = 0$
1	1	1	$z \leq 1$ $z \leq 1$ $z \geq 1$ $z \in \{0,1\}$	$z = 1$

MULTIPLICATION OF BINARY AND CONTINUOUS VARIABLES

- Non-linear constraint: $z = x \times y$ with $x \in \{0, 1\}, y \in R$
- Transformation:
  - $z \leq y$
  - $z \leq M \times x$
  - $M(x - 1) + y \leq z$
  - M is large-value

x	y	$x \cdot y$	Constraints	Imply
0	$m: 0 \leq m \leq u$	0	$z \leq m$ $z \leq 0$ $z \geq m - u$ $z \geq 0$	$z = 0$
1	$m: 0 \leq m \leq u$	m	$z \leq m$ $z \leq u$ $z \geq m$ $z \geq 0$	$z = m$

MAXIMUM/MINIMUM OPERATORS

- Non-linear constraint: $z = \max(x, y)$ with $x, y \in R$
- Transformation:
  - $z \geq y$
  - $z \geq x$
  - $z \leq M \times t + x$
  - $z \leq M \times (1 - t) + y$
  - $t \in \{0,1\}$

ABSOLUTE VALUE FUNCTION

- Absolute constraint: $|f(x)| \leq z$
  - Transformation:
    - $f(x) \leq z$
    - $-f(x) \leq z$
- Absolute constraint: $|f(x)| \geq z$
  - Transformation
    - $f(x) \geq z - t \times M$
    - $f(x) \leq -z + (1 - t) \times M$
    - $t \in \{0,1\}$
- Absolute constraint: $|f(x)| = z$
  - Transformation
    - $f(x) = z - 2 \times t \times z$
    - $t \in \{0,1\}$

CONTENT

- Introduction to integer programming
  - Relaxation and bound
  - Branch and bound
  - Cutting plane
  - Integer rounding
  - Gomory cut
  - Schema of branch-and-cut algorithm
  - OR-Tools
- Some techniques to linearize a constraint
- **Exercises**
  - **Balanced class teacher assignment**
  - Blood type correction problem
  - TSP
  - Facility location
  - MultiCast Routing Problem
  - Capacitated Vehicle Routing Problem

Balanced Class Teacher Assignment

- There are n classes labeled $1, 2, \dots, n$ that have already been scheduled in a timetable, and they need to be assigned to m teachers labeled $1,2, \dots, m$
- Each class i has $T(i)$, a list of teachers who can teach it ($i = \{1, \dots, n\}$), and $c(i)$, the number of credits for the subject of that class.
- Since the timetable has been pre-arranged, there exists a set Q of pairs of classes (i, j) that are scheduled at the same time (these two classes cannot be assigned to the same teacher).
- Find an assignment of classes to teachers such that the maximum total number of credits assigned to any one teacher is minimized.

Balanced Class Teacher Assignment

- Example

Class	0	1	2	3	4	5	6	7	8	9	10	11	12
Credit	3	3	4	3	4	3	3	3	4	3	3	4	4

Teacher	List of classes that the teach can teach
0	0, 2, 3, 4, 8, 10
1	0, 1, 3, 5, 6, 7, 8
2	1, 2, 3, 7, 9, 11, 12

List Q

0	2
0	4
0	8
1	4
1	10
3	7
3	9
5	11
5	12
6	8
6	12

41

Balanced Class Teacher Assignment

- Example

Class	0	1	2	3	4	5	6	7	8	9	10	11	12
Credit	3	3	4	3	4	3	3	3	4	3	3	4	4

Teacher	List of classes that the teach can teach
0	0, 2, 3, 4, 8, 10
1	0, 1, 3, 5, 6, 7, 8
2	1, 2, 3, 7, 9, 11, 12

Assignment solution

Teacher	List of classes assigned to the teacher	Total credits
0	2, 4, 8, 10	15
1	0, 1, 3, 5, 6	15
2	7, 9, 11, 12	14

List Q

0	2
0	4
0	8
1	4
1	10
3	7
3	9
5	11
5	12
6	8
6	12

42

Balanced Class Teacher Assignment

- Input:
  - Set of classes: $S = \{1, \dots, n\}$
  - Set of teachers: $T = \{1, \dots, m\}$
  - Set of conflict classes: $Q = \{(i_1, j_1), \dots, (i_k, j_k) | i_1, \dots, i_k, j_1, \dots, j_k \in C, i_t \neq j_t \ \forall t \in \{1, \dots, k\}\}$
  - Set of teachers who can give some class $T = \{T_1, \dots, T_n\}$, in which T_i is the set of teachers who can give the class i $(i \in \{1, \dots, n\})$
  - For each class i $(i \in \{1, \dots, n\}), c(i)$ is its number of credit $(c(i) \in N)$
- Variables:
  - Binary variable x_{ij} $(i \in C, j \in T)$ is equal to 1 if the class i is assigned to the teacher j, otherwise the value of this variable is equal to 0;
  - Integral variable $maxcredit$ represents the maximum number of credits for a teacher;
- Domains of variables:
  - $D(x_{ij}) = \{0, 1\}, \forall\, i \in C, j \in T$
  - $D(maxcredit) = \{0, \dots, \sum_{i \in C} c(i)\}$
- Constraints:
  - Each class is assigned to one teacher $\sum_{j \in T_i} x_{ij} = 1, \ \forall\, i \in C$
  - Teacher is not assigned to a class that he cannot teach $x_{ij} = 0, \forall\, i \in C, j \notin T_i$
  - Teacher cannot give two classes in the conflict set $x_{i_1 j} + x_{i_2 j} \leq 1, \forall\, j \in T, (i_1, i_2) \in Q$
  - Relation between variable $maxcredit$ and workload of teacher $\sum_{i \in C} c(i) x_{ij} \leq maxcredit\,, \forall\, j \in T$
- Objective: $Minimize\ maxcredit$

OR-Tools

```
def create_data_model():
    """Stores the data for the problem."""
    data = {}
    #Number of teachers
    data['M'] = 3

    #Number of classes
    data['N'] = 13

    #Numbers of class credits
    data['credit'] = [3, 3, 4, 3, 4, 3, 3, 3, 4, 3, 3, 4, 4 ]

    #Classes can be given by teachers
    data['teach_class'] = [[ 1, 0, 1, 0, 1, 0, 0, 0, 1, 0, 1, 0, 0 ],
                           [1, 1, 0, 1, 0, 1, 1, 1, 1, 0, 0, 0, 0 ],
                           [0, 1, 1, 1, 0, 0, 0, 1, 0, 1, 0, 1, 1 ]]
```

OR-Tools

```
#Conflicts of classes
data['class_conflict'] =[[0, 1, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0 ],
                         [1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0 ],
                         [1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 ],
                         [0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0 ],
                         [1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 ],
                         [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1 ],
                         [0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1 ],
                         [0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0 ],
                         [1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0 ],
                         [0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0 ],
                         [0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 ],
                         [0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0 ],
                         [0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0 ]]

return data
```

OR-Tools

```
def main():

    #Get data input
    data = create_data_model()

    # Create the mip solver with the SCIP backend.
    solver = pywraplp.Solver.CreateSolver('SCIP')


    ##################### VARIABLES ##############################################
    #x[i,j] = 1 if teacher i teaches class j, if not x[i,j]= 0
    x = {}
    for i in range(data['M']):
        for j in range(data['N']):
            x[i,j] = solver.IntVar(0, 1, 'x['+ str(i) + ','+ str(j) +']')

    #load[i] is total credit of classes of teacher i
    load = {}
    for i in range(data['M']):
        load[i] = solver.IntVar(0, sum(data['credit']), 'load['+ str(i) + ']')

    #max workload
    maxLoad = solver.IntVar(0, sum(data['credit']), 'maxLoad')

    print('Number of variables =', solver.NumVariables())
```

OR-Tools

```
###################### CONSTRAINTS ########################################

#Load of teachers and max load
for i in range(data['M']):
    solver.Add(solver.Sum([x[i, j]*data['credit'][j] for j in range(data['N'])]) == load[i])

    cons1 = solver.Constraint(0, solver.infinity())
    cons1.SetCoefficient(load[i], -1)
    cons1.SetCoefficient(maxLoad, 1)

# Class is assigned to teacher if he can teach
for i in range(data['M']):
    for j in range(data['N']):
        cons2 = solver.Constraint(0, data['teach_class'][i][j])
        cons2.SetCoefficient(x[i,j], 1)

# Two classes assigned to a teacher must be not conflict to each other
for i in range(data['M']):
    for j1 in range(data['N']):
        for j2 in range(data['N']):
            if data['class_conflict'][j1][j2] == 1:
                cons3 = solver.Constraint(0, 1)
                cons3.SetCoefficient(x[i,j1],1)
                cons3.SetCoefficient(x[i,j2],1)

# Classes must be assigned all to teachers
for i in range(data['N']):
    cons4 = solver.Constraint(1,1)
    for j in range(data['M']):
        cons4.SetCoefficient(x[j,i], 1)
```

OR-Tools

```
    ########################## OBJECTIVE ##################################
    solver.Minimize(maxLoad)

    status = solver.Solve()

    ########################## PRINT SOLUTION #############################
    if status == pywraplp.Solver.OPTIMAL:
        print('Solution:')
        print('Objective value =', solver.Objective().Value())
        for i in range(data['M']):
            for j in range(data['N']):
                print(x[i,j].name(), ' = ', x[i,j].solution_value())

        for i in range(data['M']):
            print(load[i].name(), ' = ', load[i].solution_value())
    else:
        print('The problem does not have an optimal solution.')


if __name__ == '__main__':
    main()
```

CONTENT

- Introduction to integer programming
  - Relaxation and bound
  - Branch and bound
  - Cutting plane
  - Integer rounding
  - Gomory cut
  - Schema of branch-and-cut algorithm
  - OR-Tools
- Some techniques to linearize a constraint
- **Exercises**
  - Balanced class teacher assignment
  - **Blood type correction problem**
  - TSP
  - Facility location
  - MultiCast Routing Problem
  - Capacitated Vehicle Routing Problem

BLOOD TYPE CORRECTION PROBLEM

- In Vietnam's National Resident Database, information about each citizen's blood type is recorded. Due to the manual data collection process, errors exist in some records within the database. The objective of this exercise is to determine the minimum number of records that must be corrected to ensure the database's accuracy and consistency. The problem is defined as follows: Consider a family with multiple individuals, where each individual is assigned a blood type (O, A, B, or AB) and linked to two parents. Note that information about an individual's parents may sometimes be missing. Using the rules of blood type inheritance and the provided family information, answer the question: What is the smallest number of individuals in the family whose blood types must be changed to ensure that the blood type inheritance rules are satisfied for the entire family?
- Question 1: Propose Mixed Integer Program for the problem
- Question 2: Implement the MIP in Hustack (programming.daotao.ai)

Blood types of parent	Possible blood types of the child
A, A	O, A
B, B	O, B
O, O	O
A, B	O, A, B, AB
A, O	A, O
A, AB	A, B, AB
B, AB	A, B, AB
O, B	O, B
O, AB	A, B
AB, AB	A, B, AB

INTEGER PROGRAM FOR BLOOD TYPE CORRECTION PROBLEM

- From the blood type combination rules presented, we derive the following four rules. Given three individuals X, Y, and Z, where X and Y are the parents of Z:
- **Rule 1**: If the blood type of Z is O, then the blood types of both X and Y must not be AB.
- **Rule 2**: If the blood type of Z is A, then the blood type of either X or Y must be A or AB.
- **Rule 3**: If the blood type of Z is B, then the blood type of either X or Y must be B or AB.
- **Rule 4**: If the blood type of Z is AB, then the blood type of either X or Y must be AB and both X and Y are different from O; or the blood types of X and Y must be A and B; or B and A, respectively.

Blood types of parent	Possible blood types of the child
A, A	O, A
B, B	O, B
O, O	O
A, B	O, A, B, AB
A, O	A, O
A, AB	A, B, AB
B, AB	A, B, AB
O, B	O, B
O, AB	A, B
AB, AB	A, B, AB

DECISION VARIABLES

- For each individual i ($i \in \{1, \ldots, n\}$), we define five binary variables $x_i^1, x_i^2, x_i^3, x_i^4$, and y_i, where:
  - x_i^1 (resp. x_i^2, x_i^3, and x_i^4) is equal to 1 if their blood type is O (resp. A, B, and AB), and is equal to 0 otherwise;
  - y_i is equal to 1 if the blood type of individual i must be changed.
- For each married couple $\langle i, j \rangle$, we define a pair of binary variables z_{ij} and z_{ji}, where:
  - $z_{ij} = 1$ if the blood type of i is A and the blood type of j is B; otherwise, $z_{ij} = 0$.
  - $z_{ji} = 1$ if the blood type of i is B and the blood type of j is A; otherwise, $z_{ji} = 0$.

CONSTRAINTS

$$x_i^1 = 1, \forall i \in \{1, \ldots, n\}, l_i = 1, b_i = \mathrm{O} \quad (1)$$

$$x_i^2 = 1, \forall i \in \{1, \ldots, n\}, l_i = 1, b_i = \mathrm{A} \quad (2)$$

$$x_i^3 = 1, \forall i \in \{1, \ldots, n\}, l_i = 1, b_i = \mathrm{B} \quad (3)$$

$$x_i^4 = 1, \forall i \in \{1, \ldots, n\}, l_i = 1, b_i = \mathrm{AB} \quad (4)$$

$$x_i^1 + x_i^2 + x_i^3 + x_i^4 = 1, \ \forall i \in \{1, \ldots, n\} \quad (5)$$

$$z_{ij} \leq x_i^1, \ \forall i, j \in \{1, \ldots, n\}, \quad (6)$$

$$z_{ij} \leq x_j^2, \ \forall i, j \in \{1, \ldots, n\}, \quad (7)$$

$$x_i^1 + x_j^2 \leq z_{ij} + 1, \ \forall i, j \in \{1, \ldots, n\}, \quad (8)$$

$$z_{ji} \leq x_i^2, \ \forall i, j \in \{1, \ldots, n\}, \quad (9)$$

$$z_{ji} \leq x_j^1, \ \forall i, j \in \{1, \ldots, n\}, \quad (10)$$

$$x_i^2 + x_j^1 \leq z_{ji} + 1, \ \forall i, j \in \{1, \ldots, n\}, \quad (11)$$

$$2 \times (1 - x_i^1) - x_{m_i}^4 - x_{f_i}^4 \geq 0, \ \forall i, f_i, m_i \in \{1, \ldots, n\} \quad (12)$$

$$(1 - x_i^2) + x_{m_i}^2 + x_{f_i}^2 + x_{m_i}^4 + x_{f_i}^4 \geq 1, \ \forall i, f_i, m_i \in \{1, \ldots, n\} \quad (13)$$

$$(1 - x_i^3) + x_{m_i}^3 + x_{f_i}^3 + x_{m_i}^4 + x_{f_i}^4 \geq 1, \ \forall i, f_i, m_i \in \{1, \ldots, n\} \quad (14)$$

$$(1 - x_i^4) + x_{m_i}^4 + x_{f_i}^4 - (x_{m_i}^1 + x_{f_i}^1) + z_{m_i f_i} + z_{f_i m_i} \geq 1, \ \forall i, f_i, m_i \in \{1, \ldots, n\} \quad (15)$$

$$y_i = x_i^2 + x_i^3 + x_i^4, \forall i \in \{1, \ldots, n\}, b_i = O \quad (16)$$

$$y_i = x_i^1 + x_i^3 + x_i^4, \forall i \in \{1, \ldots, n\}, b_i = A \quad (17)$$

$$y_i = x_i^1 + x_i^2 + x_i^4, \forall i \in \{1, \ldots, n\}, b_i = B \quad (18)$$

$$y_i = x_i^1 + x_i^2 + x_i^3, \forall i \in \{1, \ldots, n\}, b_i = AB \quad (19)$$

CONSTRAINTS

- Constraints (1), (2), (3), and (4) indicate that if the correctness level of individual i is 1, then their blood type is fixed.
- Constraints (5) ensure that each individual has only one blood type.
- Constraints (6), (7), and (8) ensure that $z_{ij} = 1$ if and only if $x_i^1 = 1$ and $x_j^2 = 1$. This means that if the blood type of individual i is A and the blood type of individual j is B, then $z_{ij} = 1$.
- Constraints (9), (10), and (11) ensure that $z_{ji} = 1$ if and only if $x_i^2 = 1$ and $x_j^1 = 1$. This means that if the blood type of individual i is B and the blood type of individual j is A, then $z_{ji} = 1$.
- Constraints (12) ensure that if the blood type of individual i is O ($x_i^1 = 1$), then the blood types of both their parents must not be AB ($x_{m_i}^4 = x_{f_i}^4 = 0$). These represent ***Rule 1***.
- Constraints (13) ensure that if the blood type of individual i is A ($x_i^2 = 1$), then the blood type of either their father or mother is either A or AB ($x_{m_i}^2 + x_{f_i}^2 + x_{m_i}^4 + x_{f_i}^4 \geq 1$). These represent ***Rule 2***.

CONSTRAINTS

- Constraints (14) ensure that if the blood type of individual i is B ($x_i^3 = 1$), then the blood type of at least one of the two parents is either B or AB ($x_{m_i}^3 + x_{f_i}^3 + x_{m_i}^4 + x_{f_i}^4 \geq 1$). These represent ***Rule 3***.
- Constraints (15) and (5) ensure that if the blood type of individual i is AB ($x_i^4 = 1$), then the blood type of at least one of the two parents is AB and the blood type of no one is O; or blood types of two parents are A and B. These represent ***Rule 4***.
- Constraints (16), (17), (18), and (19) state that if the blood type of individual i is modified, then the value of the penalty variable y_i must be equal to 1.
- The objective function of the model is to minimize the number of changes.

OBJECTIVE

$$\text{Minimize } \sum_{i=1}^{n} y_i$$

CONTENT

- Introduction to integer programming
  - Relaxation and bound
  - Branch and bound
  - Cutting plane
  - Integer rounding
  - Gomory cut
  - Schema of branch-and-cut algorithm
  - OR-Tools
- Some techniques to linearize a constraint
- **Exercises**
  - Balanced class teacher assignment
  - Blood type correction problem
  - **TSP**
  - Facility location
  - MultiCast Routing Problem
  - Capacitated Vehicle Routing Problem

Traveling Salesman Problem

- Input:
  - n number of cities
  - $c(i,j)$ traveling cost from the city i to the city j
- Variables:
  - Binary variable x_{ij} with $i,j \in \{1,\dots,n\}, i \neq j$ is equal to 1 if the traveler go from the city i to the city j in the optimal plan, otherwise the value of this variable is equal to 0;
- Constraints:
  - For each city, the traveler goes in once and goes out once

$$\sum_{j\in\{1,\dots,n\}} x_{ij} = \sum_{j\in\{1,\dots,n\}} x_{ji} = 1, \forall i \in \{1,\dots,n\}$$

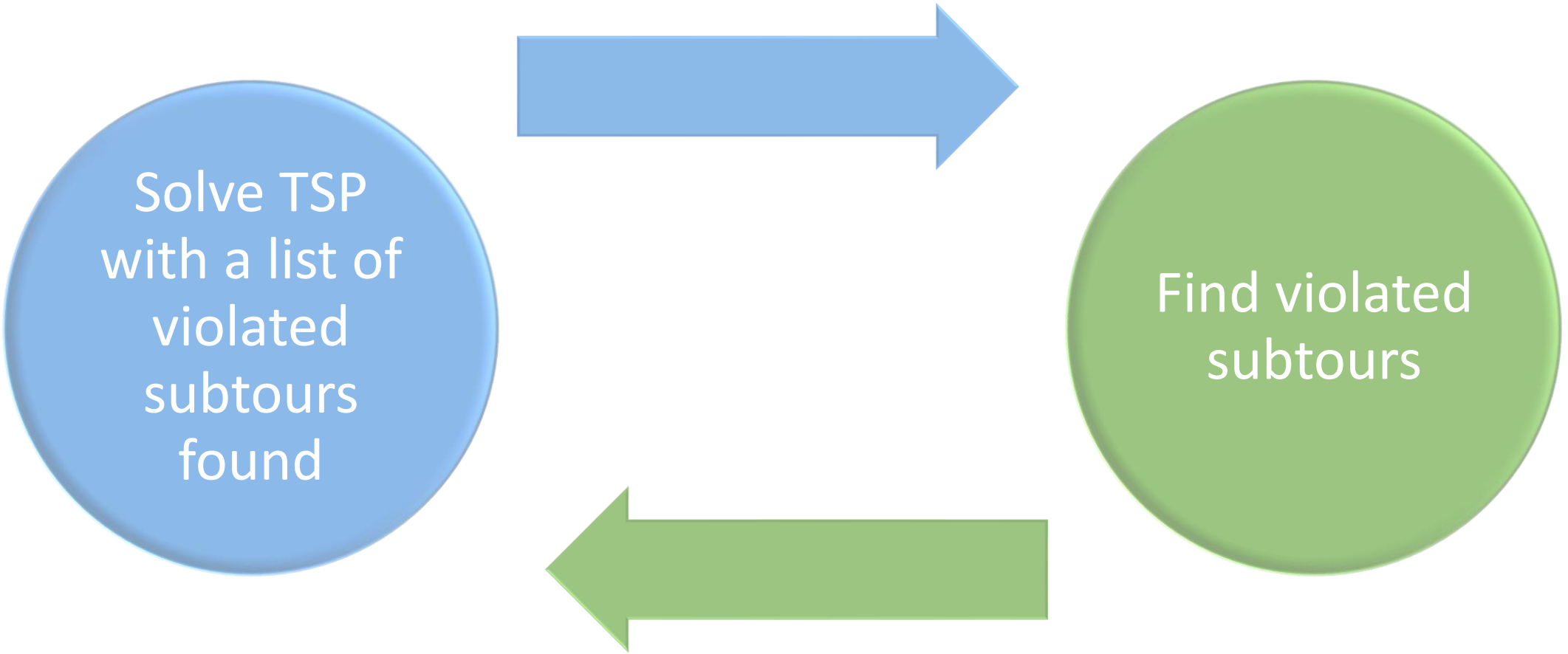
  - No subtour

$$\sum_{i,j\in S, i\neq j} x_{ij} \leq |S| - 1, \forall S \subset \{1,\dots,n\}$$

Remove them?

- Objective: $Minimize\ \sum_{i,j\in\{1,\dots,n\}, i\neq j} c(i,j)x_{ij}$

Traveling Salesman Problem



TSP

```
n = 5
distance  = [[0, 1, 2, 4, 5],
             [1, 0, 3, 0, 0],
             [2, 3, 0, 1, 0],
             [4, 0, 1, 0, 4],
             [5, 0, 0, 4, 0]
            ]
sec_lst = list()

from ortools.linear_solver import pywraplp

def MIP_Solver():
    global sec_lst

    # Create the mip solver with the CP-SAT backend.
    solver = pywraplp.Solver.CreateSolver("SCIP")

    # List of decision variables
    X = list()
    for i in range(n):
        tmp = list()
        for j in range(n):
            tmp.append(solver.IntVar(0, 1, "x[" + str(i) + "][" + str(j)+"]"))
        X.append(tmp)
```

```
# Constrains
for i in range(n):
    for j in range(n):
        if distance[i][j] <= 0:
            solver.Add(X[i][j] == 0)

for i in range(n):
    solver.Add(X[i][i] == 0)
    flow_val1 = 0
    flow_val2 = 0
    for j in range(n):
        flow_val1 += X[j][i]
        flow_val2 += X[i][j]
    solver.Add(flow_val1 == flow_val2)
    solver.Add(flow_val1 == 1)

#Subtour
for sec in sec_lst:
    var = 0
    print("SEC: ", sec)
    for i in range(len(sec)):
        u = sec[i]
        for j in range(i+1, len(sec)):
            v = sec[j]
            var += X[u][v] + X[v][u]
    solver.Add(var <= len(sec) - 1)
```

TSP

```
# Objective
obj = 0
for i in range(n):
    for j in range(n):
        obj += X[i][j] * distance[i][j]

solver.Minimize(obj)

#print(f"Solving with {solver.SolverVersion()}")
status = solver.Solve()

if status == pywraplp.Solver.OPTIMAL:
    #print("Solution:")
    #print("Objective value =", solver.Objective().Value())

    next_node = [-1] * n
    for i in range(n):
        for j in range(n):
            if X[i][j].solution_value() > 0:
                next_node[i] = j
                print(i, "=>", j)
                break
```

```
    mark = [0] * n
    new_sec_lst = list()

    for i in range(n):
        if mark[i] == 0:
            sec = list()
            next = i
            while True:
                next = next_node[next]
                if mark[next] == 0:
                    sec.append(next)
                    mark[next] = 1
                else:
                    new_sec_lst.append(sec)
                    break

    if len(new_sec_lst) > 1:
        for sec in new_sec_lst:
            sec_lst.append(sec)

        return None
    else:
        return solver.Objective().Value()
else:
    return None
```

```
while True:
    res = MIP_Solver()
    print("length of sec: ", len(sec_lst))

    if res is not None:
        print(round(res))
        break
    else:
        print(-1)
```

CONTENT

- Introduction to integer programming
  - Relaxation and bound
  - Branch and bound
  - Cutting plane
  - Integer rounding
  - Gomory cut
  - Schema of branch-and-cut algorithm
  - OR-Tools
- Some techniques to linearize a constraint
- **Exercises**
  - Balanced class teacher assignment
  - Blood type correction problem
  - TSP
  - **Facility location problem**
  - MultiCast routing problem
  - Capacitated vehicle routing problem

Facility Location

- There are *M* sites 1, 2, …, *M* that can be used to open facility for servicing *N* customers 1, 2, …, *N*.
  - *f*(*i*) is the cost for opening the site *i*
  - *Q*(*i*) is the capacity of site *i* (maximum amount of good it can serve customers)
  - *c*(*i*,*j*) is the cost for transporting unit of good from site *i* to customer *j*
  - *d*(*j*) is the total demand (amount of goods) of customer *j*
- Compute a planning (which site to be opened and amount of good each opened site serves a customer) such that
  - Capacity constraint is satisfied
  - Total cost is minimal

Facility Location

- Decision variables
  - $Y(i)$ – binary variable, $Y(i) = 1$ means that the site i is opened, and $Y(i) = 0$, otherwise
  - $X(i,j)$ – amount of good site i serves customer j
- Constraints
  - $\sum_{i=1}^{M} X(i,j) = d(j), \forall\, j = 1,\ldots, N$
  - $\sum_{j=1}^{N} X(i,j) \le Q(i)Y(i), \forall\, i = 1,\ldots, M$
  - $0 \le X(i,j) \le d(j)Y(i), \forall\, i = 1,\ldots, M, \forall\, j = 1,\ldots, N$
- Objective functions

$$\sum_{i=1}^{M} f(i)Y(i) + \sum_{i=1}^{M}\sum_{j=1}^{N} c(i,j)X(i,j) \rightarrow \min$$

CONTENT

- Introduction to integer programming
  - Relaxation and bound
  - Branch and bound
  - Cutting plane
  - Integer rounding
  - Gomory cut
  - Schema of branch-and-cut algorithm
  - OR-Tools
- Some techniques to linearize a constraint
- **Exercises**
  - Balanced class teacher assignment
  - Blood type correction problem
  - TSP
  - Facility location
  - **Multicast routing problem**
  - Capacitated vehicle routing problem

Multicast Routing Problem

- Given a network V = {1,...,N} is the set of nodes, $E \subseteq V^2$ is the set of links between nodes. A node $s \in V$ is the source node which will transmit a package to others nodes. A node receiving the package can continue transmit this package to adjacent nodes.
  - t(i,j) and c(i,j) are transmission time and transmission cost when transmitting the package from node i to node j
- Compute the set of links used for broadcasting the package from the source node to all other nodes such that
  - Total transmission time from s to any node cannot exceed a given value L
  - Total transmission cost is minimal

Multicast Routing Problem

- Denote $A(i) = \{j \in V \mid (i,j) \in E\}$, M is a big constant
- Decision variables
  - Binary variable $X(i,j) = 1$ if the package is transmitted from node i to node j, and $X(i,j) = 0$, otherwise, $\forall (i,j) \in E$
  - $Y(i)$: time-point when the package arrives at node i, $\forall i \in V$
- Constraints
  - $\sum_{i \in A(j)} X(i,j) = 1, \forall j \in V \setminus \{s\}$
  - $Y(i) + t(i,j) + M(1-X(i,j)) \geq Y(j), \forall (i,j) \in E$
  - $Y(i) + t(i,j) + M(X(i,j)-1) \leq Y(j), \forall (i,j) \in E$
  - $Y(i) \leq L, \forall j \in V \setminus \{s\}$
  - $Y(s) = 0$
- Objective function to be minimized

$$f(X) = \sum_{(i,j) \in E} c(i,j) X(i,j)$$

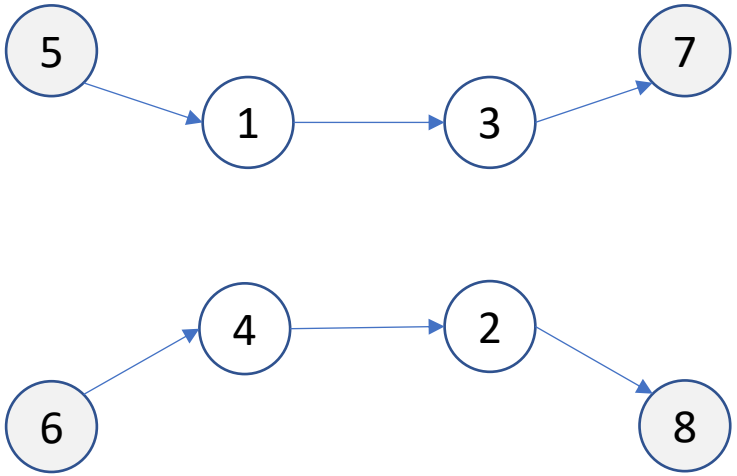
CONTENT

- Introduction to integer programming
  - Relaxation and bound
  - Branch and bound
  - Cutting plane
  - Integer rounding
  - Gomory cut
  - Schema of branch-and-cut algorithm
  - OR-Tools
- Some techniques to linearize a constraint
- **Exercises**
  - Balanced class teacher assignment
  - Blood type correction problem
  - TSP
  - Facility location problem
  - MultiCast routing problem
  - **Capacitated vehicle routing problem**

Capacitated vehicle routing problem

- A fleet of K trucks 1, 2, …, K must be scheduled to visit N customers 1, 2, …, N for collecting items
  - Customer i located at point i and requests to be collected $r(i)$ items, i = 1, 2, …, N
  - Truck k (k = 1,…, K)
    - Departs from point $N+k$ and terminates at point $N + K + k$ ($N+k$ and $N+K+k$ might refer to the central depot)
    - Has capacity $c(k)$ which is the maximum number of items it can carry at a time
  - Travel distance from point i to point j is $d(i,j)$, i,j = 1,…,$N + 2K$
- Compute the delivery solution such that the total travelling distance is minimal
  - Satisfy capacity constraints
  - Each customer is visited exactly once by exactly one truck

Capacitated vehicle routing problem



1	0	2	3	4	3	3	3	3
2	4	0	2	6	1	1	1	1
3	2	4	0	2	1	1	1	1
4	5	7	7	0	4	4	4	4
5	3	1	5	7	0	0	0	0
6	3	1	5	7	0	0	0	0
7	3	1	5	7	0	0	0	0
8	3	1	5	7	0	0	0	0

	1	2	3	4
r	4	3	6	5

	1	2
c	10	10

Capacitated vehicle routing problem

- Notations
  - $B = \{1,\ldots, N+2K\}$
  - $F_1 = \{(i, k+N) \mid i \in B, k \in \{1,\ldots,K\}\}$
  - $F_2 = \{(k+K+N, i) \mid i \in B, k \in \{1,\ldots,K\}\}$
  - $F_3 = \{(i, i) \mid i \in B\}$
  - $A = B^2 \setminus F_1 \setminus F_2 \setminus F_3$
  - $A^+(i) = \{ j \mid (i, j) \in A\}$, $A^-(i) = \{ j \mid (j, i) \in A\}$
- Decision variables
  - $X(k,i,j) = 1$ if truck k travel from point i to point j, $\forall k = 1,\ldots,K$, $(i,j) \in A$
  - $Y(k,i)$: number of items on truck k after leaving point i, $\forall k = 1,\ldots,K$, $\forall i = 1,\ldots,N+2K$
  - $Z(i)$: index of truck visiting point i, $\forall\ i = 1,2,\ldots, N+2K$

HUST

hust.edu.vn fb.com/dhbkhn

THANK YOU !